Linguagens Formais e Autômatos

Máquinas de Turing

Eduardo Furlan Miranda

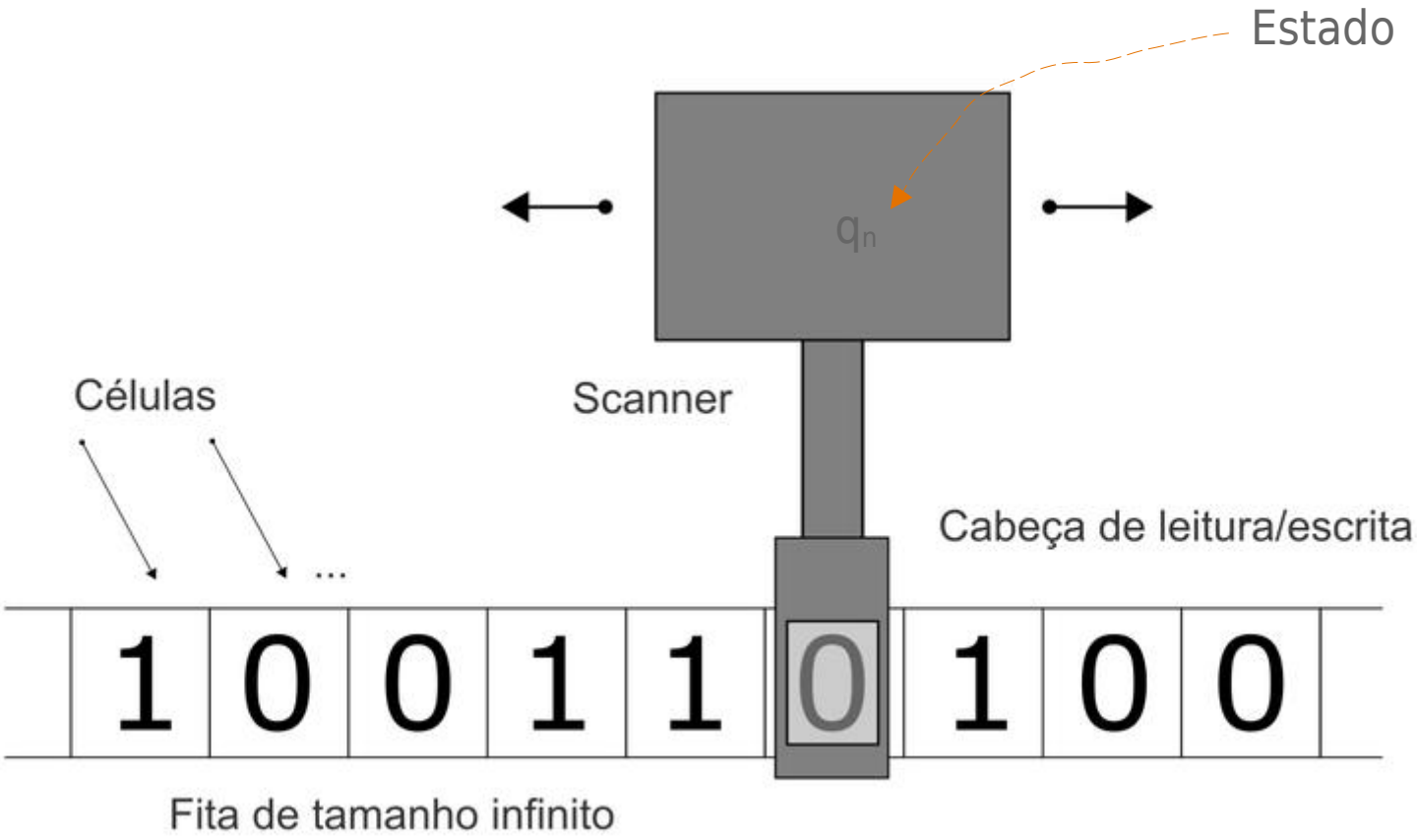
Baseado em: GARCIA, A. de V.; HAEUSLER, E. H.
Linguagens Formais e Autômatos. Londrina: EDA, 2017.

Máquina de Turing (MT)

- Modelo teórico de computação proposto por Alan Turing em 1936
- Foi concebido para formalizar a noção de *procedimento* ou *algoritmo*, estabelecendo assim o conceito de *função computável*
- Em teoria, uma MT é capaz de resolver qualquer problema que possa ser formalizado matematicamente, incluindo problemas de análise sintática
- No entanto, na prática, outras ferramentas e algoritmos são preferidos para a análise sintática, devido à maior eficiência computacional e à complexidade reduzida em comparação com a implementação direta de uma MT

MT como AF

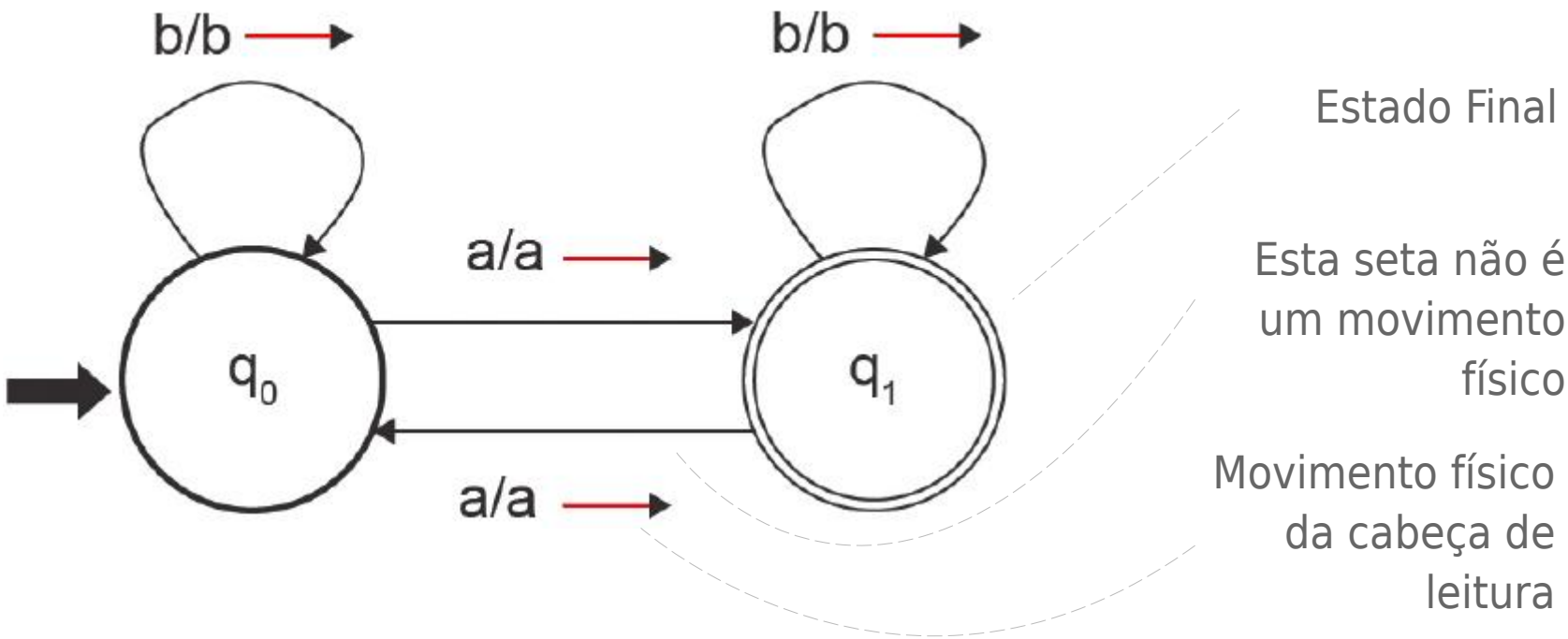
- Podemos entender a MT como um Autômato Finito (AF) com uma estrutura de dados auxiliar (uma fita)



- O dado de entrada está escrito na fita
- A cada transição a máquina pode
  - ler um símbolo do alfabeto
  - escrever sobre o mesmo
  - andar na fita para direita ou para a esquerda
  - mudar de estado

- Vamos supor que a fita é infinita para a direita
  - possui uma primeira posição
  - cada posição da fita sempre possui uma posição à sua direita
- Cada posição da fita armazena um símbolo do alfabeto da fita
- No início da execução, para uma entrada de tamanho n,
  - as n primeiras posições da fita contém os símbolos da entrada
- As demais posições da fita estão em branco
  - possuem o símbolo branco

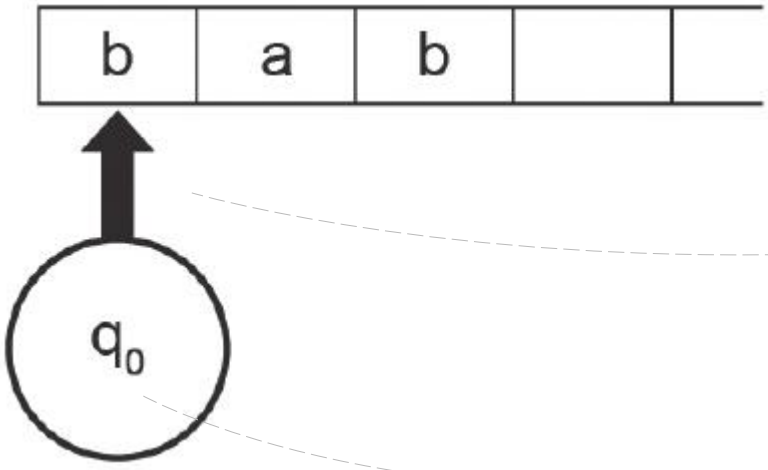
AF



- Ilustração de uma MT que reconhece a linguagem L = {w | w tem quantidade ímpar de a's }

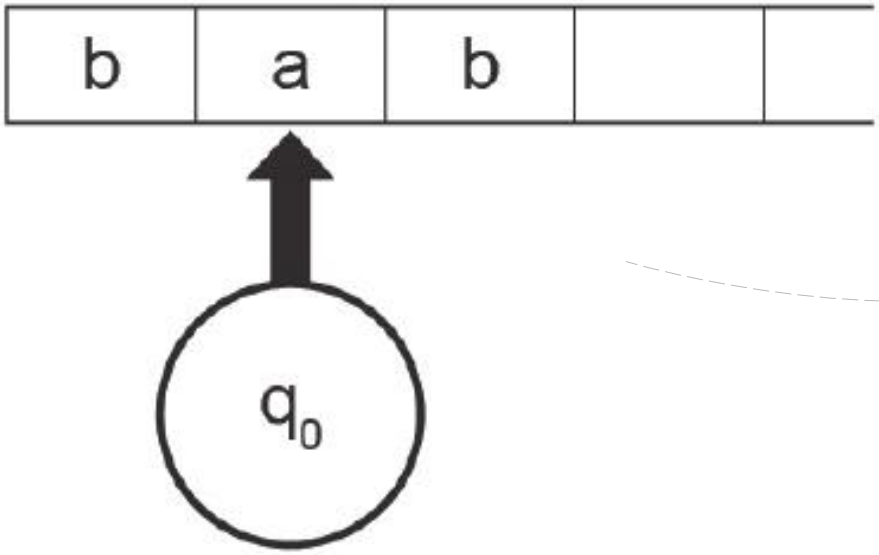
- Neste caso particular a MT anda sempre para a direita (representado pelo símbolo →)

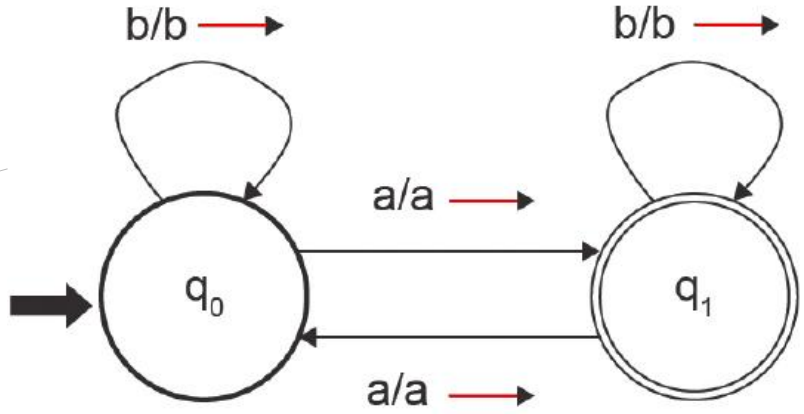
- Neste caso a máquina recebe apenas a cadeia de entrada,
  - e nenhuma informação adicional
- portanto a MT tem o mesmo poder de computação que um AF,
  - a linguagem reconhecida é uma Linguagem Regular (LR)
    - LR é uma linguagem formal que pode ser expressa usando Expressões Regulares (ER), ou seja,
      - uma linguagem produzida utilizando as operações de concatenação, união e fecho de Kleene sobre os elementos de um alfabeto
- Qualquer linguagem que pode ser reconhecida por um AF também é uma LR



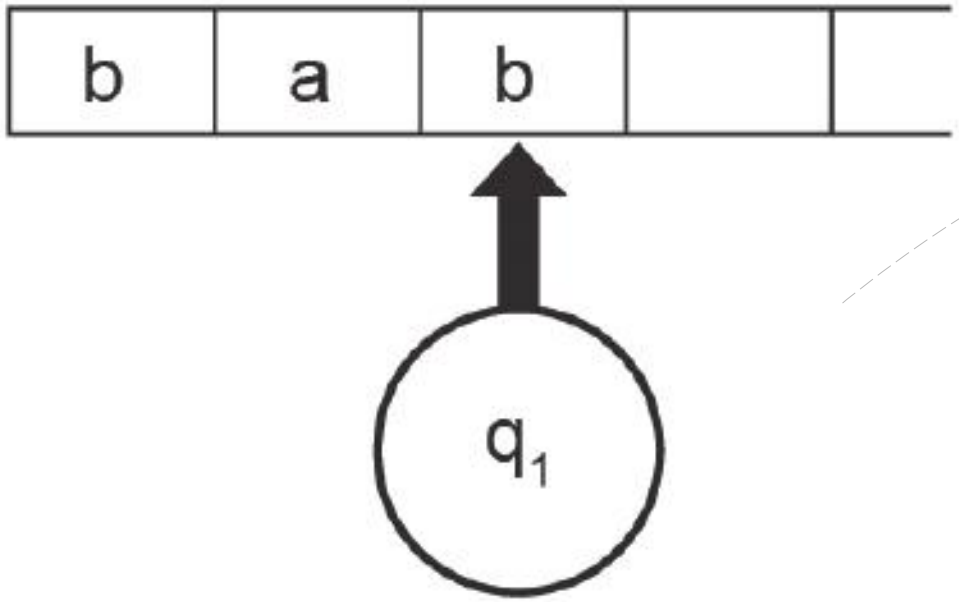
- Suponha que desejamos reconhecer a cadeia bab
- Neste caso, a cadeia estará no início da fita
- A posição da fita sendo lida pela máquina é representada pela seta vertical
  - e o estado no qual a máquina está é representado no círculo

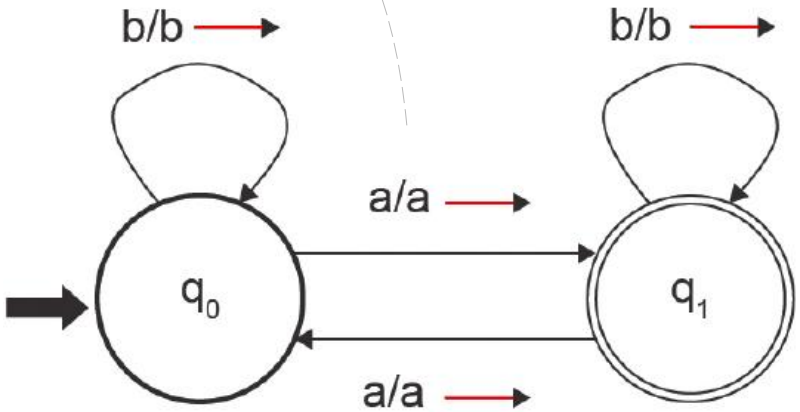
- Estando no estado q_0 e lendo b ,
  - a máquina escreve um b por cima
    - portanto não vai alterar o conteúdo da fita
- Segue para a direita e
  - continua no estado q_0



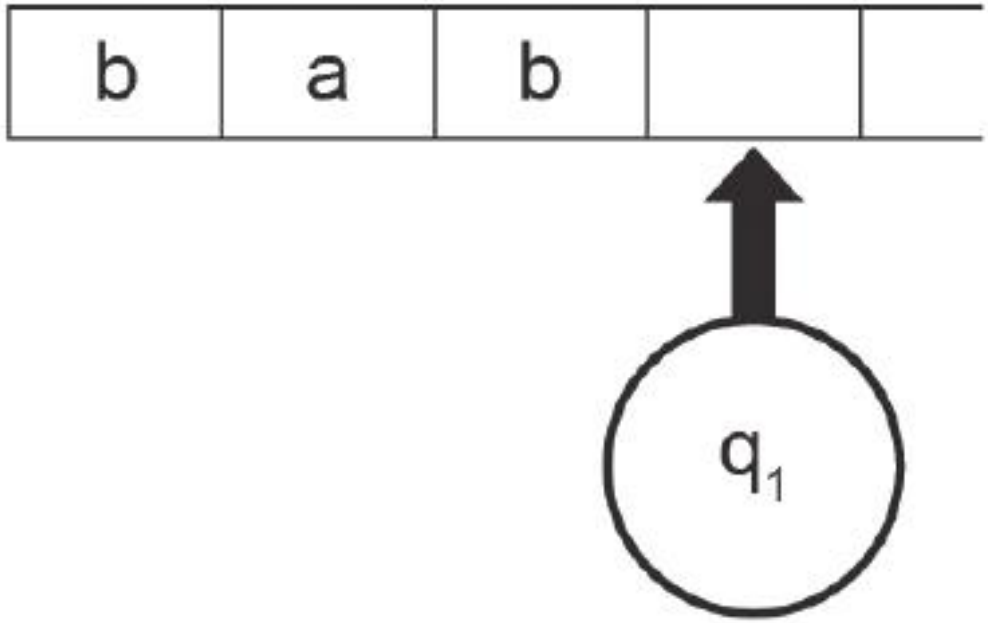


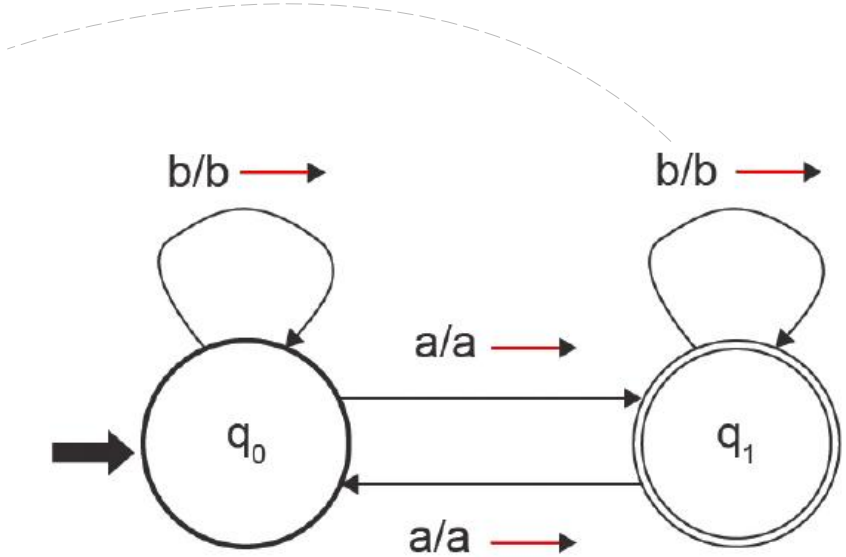
- Neste ponto, estando no estado q_0 e lendo um a, a MT irá escrever um a por cima (portanto, não vai alterar o conteúdo da fita), seguir para a direita (devido ao símbolo →) e mudar para o estado q_1





- Finalmente, estando no estado q_1 e lendo um b, a MT irá escrever um b por cima (portanto não vai alterar o conteúdo da fita), seguir para a direita (devido ao símbolo →) e permanecer no estado q_1





- Neste caso, não se especifica o que fazer quando a MT lê um branco
- Dizemos que a MT parou
- Como ela parou em um estado final (q_1), dizemos que ela reconheceu a sentença bab

MT

R	$a^n b$
LC	$a^n b^n$
SC	$a^n b^n c^n$
I	a^{2^n}

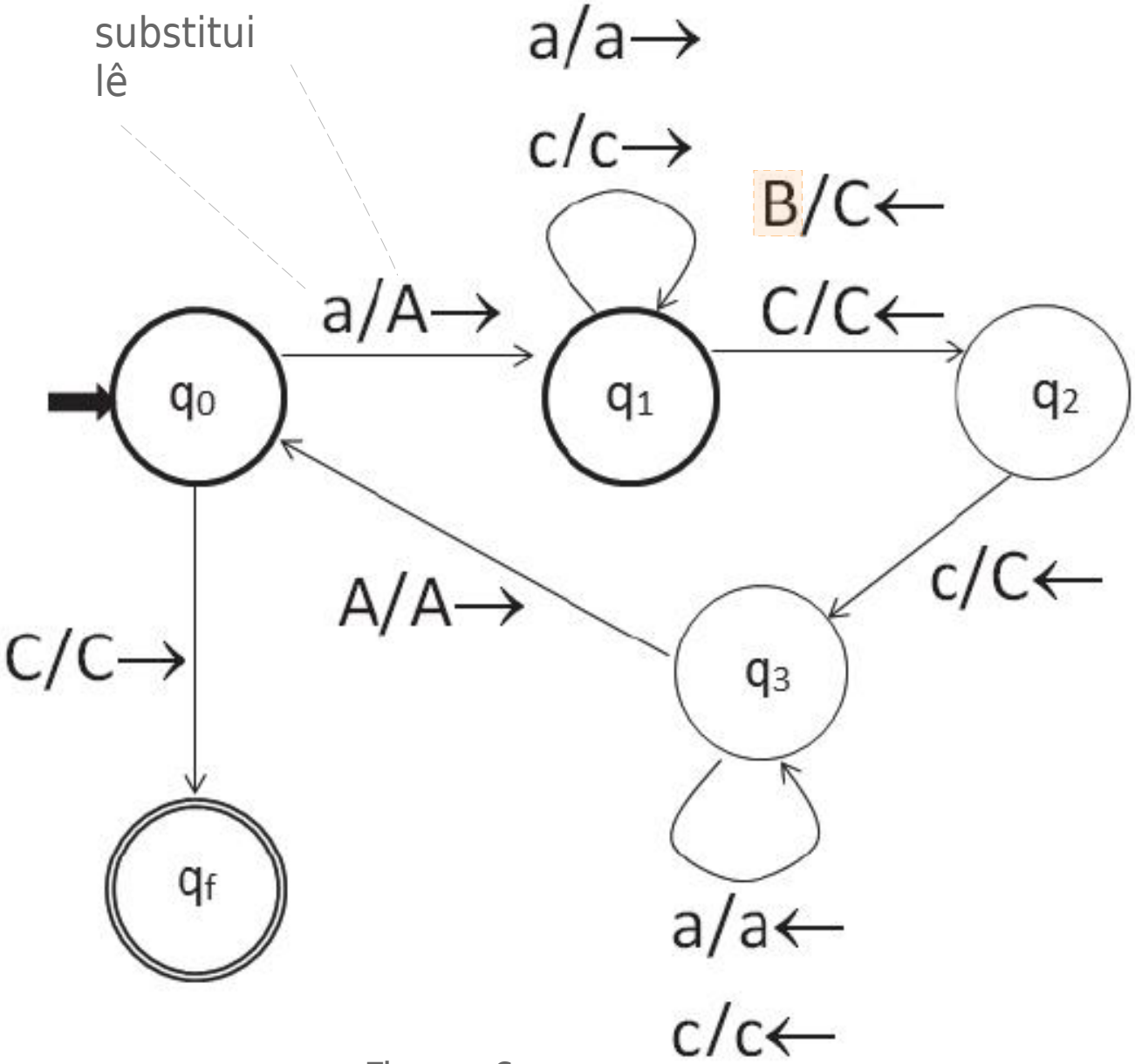


Figura 6

- MT que reconhece a linguagem $L=\{a^n c^n \mid n \geq 1\}$
- Como esta linguagem não é regular, a MT terá obrigatoriamente de se mover para a esquerda

- A técnica usada é marcar cada caractere lido:
  - cada a lido é substituído por um A e
  - cada c lido por um C

(vide slide anterior)

- O caractere branco é representado por B para facilitar a visualização

- Por convenção não é permitido escrever o B ,
  - portanto ele também é substituído por C

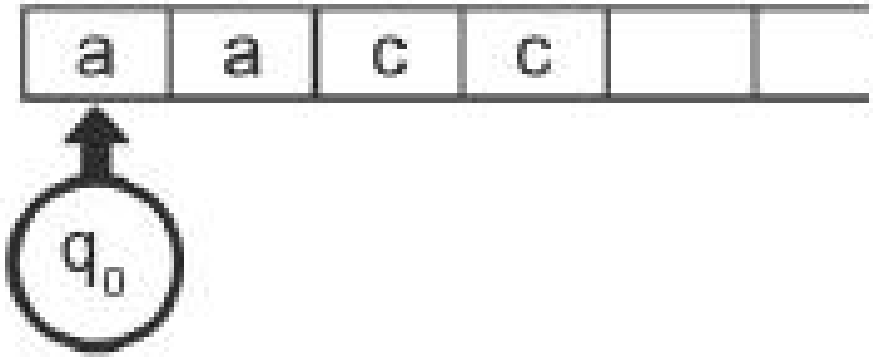
(vide slide anterior)

- Podemos representar a configuração do autômato em um dado ponto por $\alpha q \beta$, onde
  - α é o conteúdo da fita antes da cabeça de leitura
  - q é o estado no qual a máquina se encontra
  - β é o conteúdo da fita a partir da cabeça de leitura até o primeiro caractere branco (B)

- A configuração inicial da MT do slide anterior, ao reconhecer a cadeia aacc, é $q_0$aacc

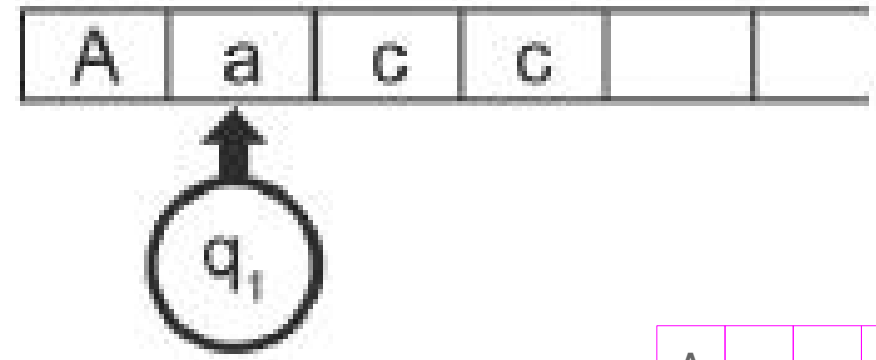
- A figura do próximo slide ilustra o reconhecimento passo a passo

1.



q_0aacc

2.



Aq_1acc

A	a	c	c	B	B

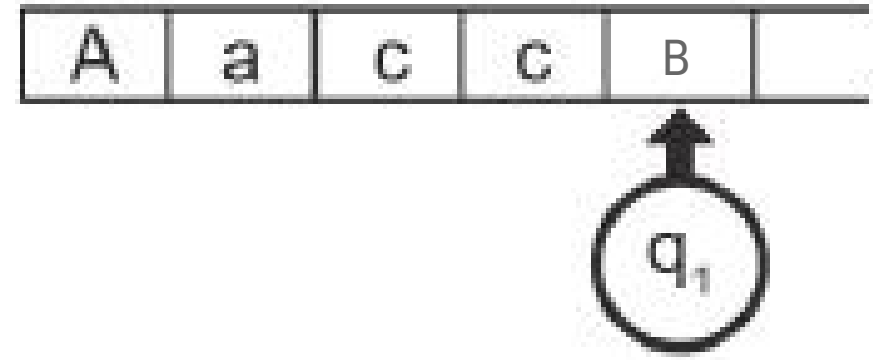
q1

... ...

A	a	c	c	B	B

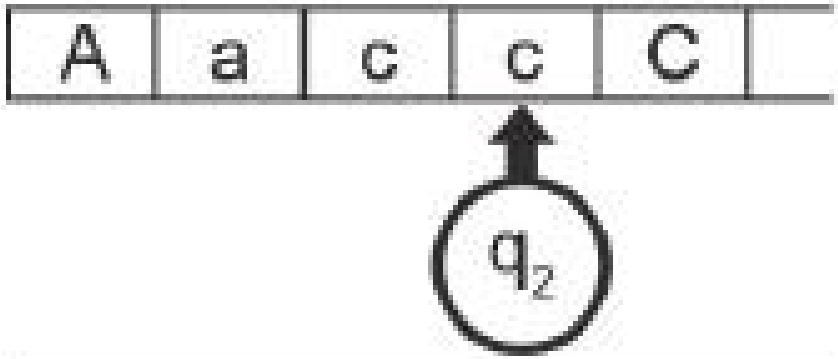
q1

5.



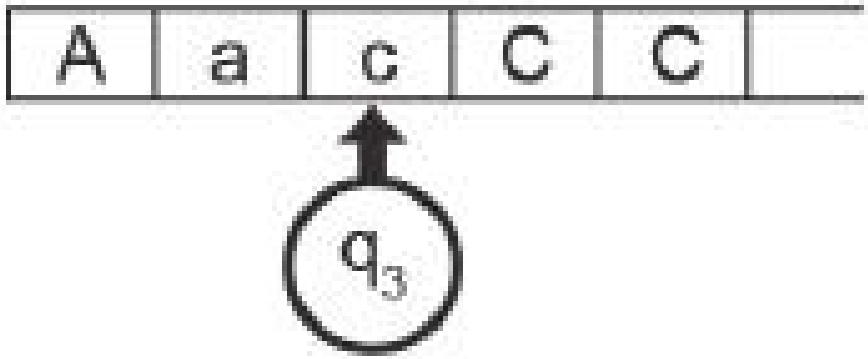
$Aaccq_1$

6.



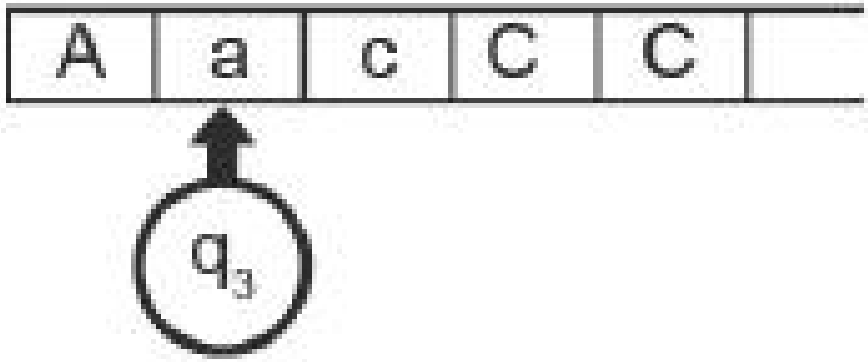
$Aacq_2cC$

7.



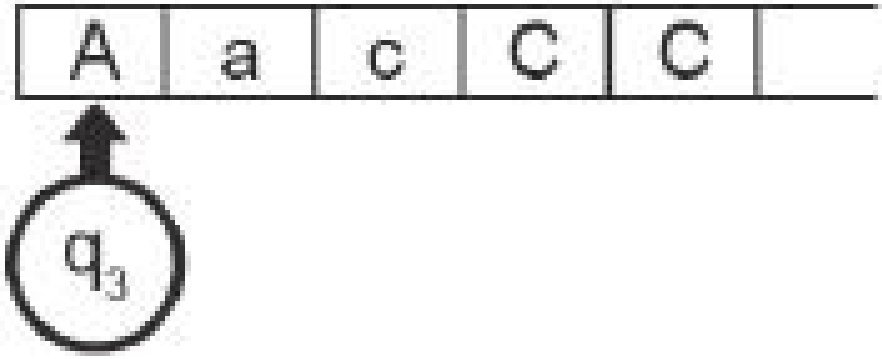
Aaq_3cCC

8.

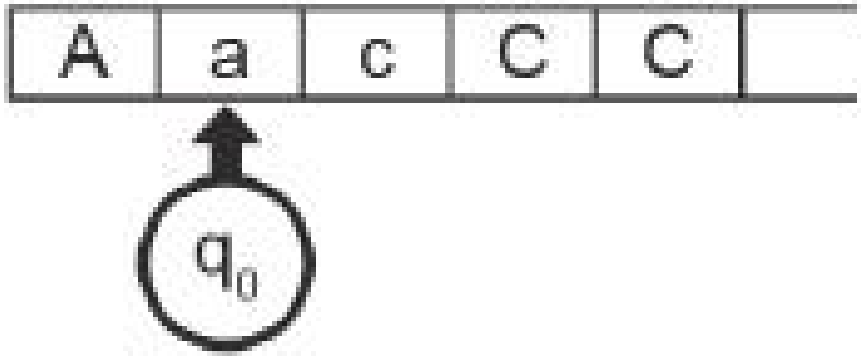


Aq_3acCC

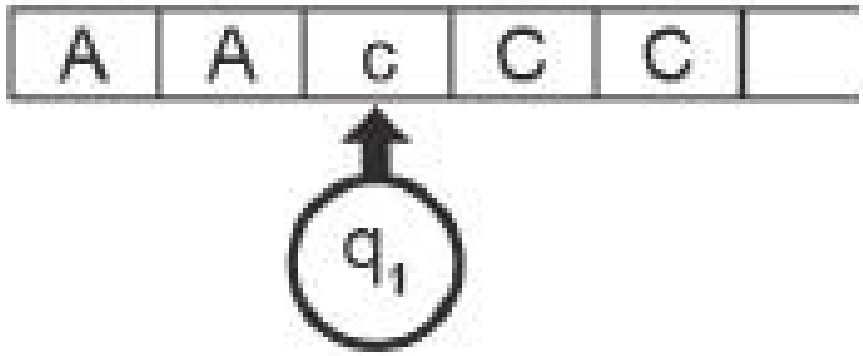
9.



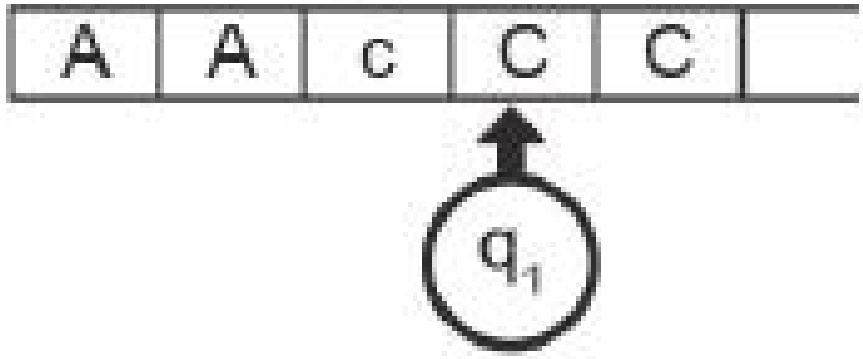
q_3AacCC

10. 

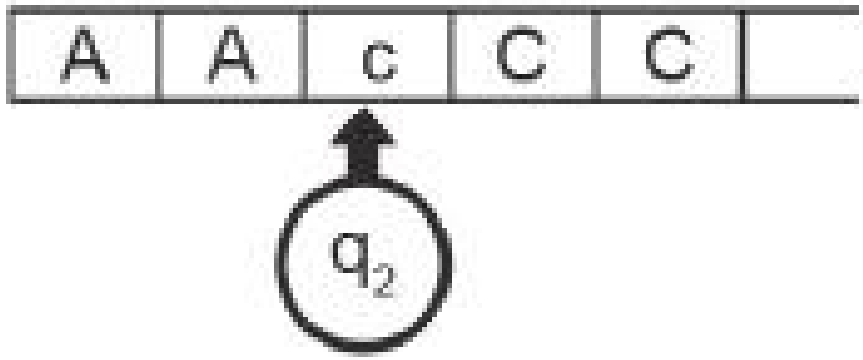
Aq_0acCC

11. 

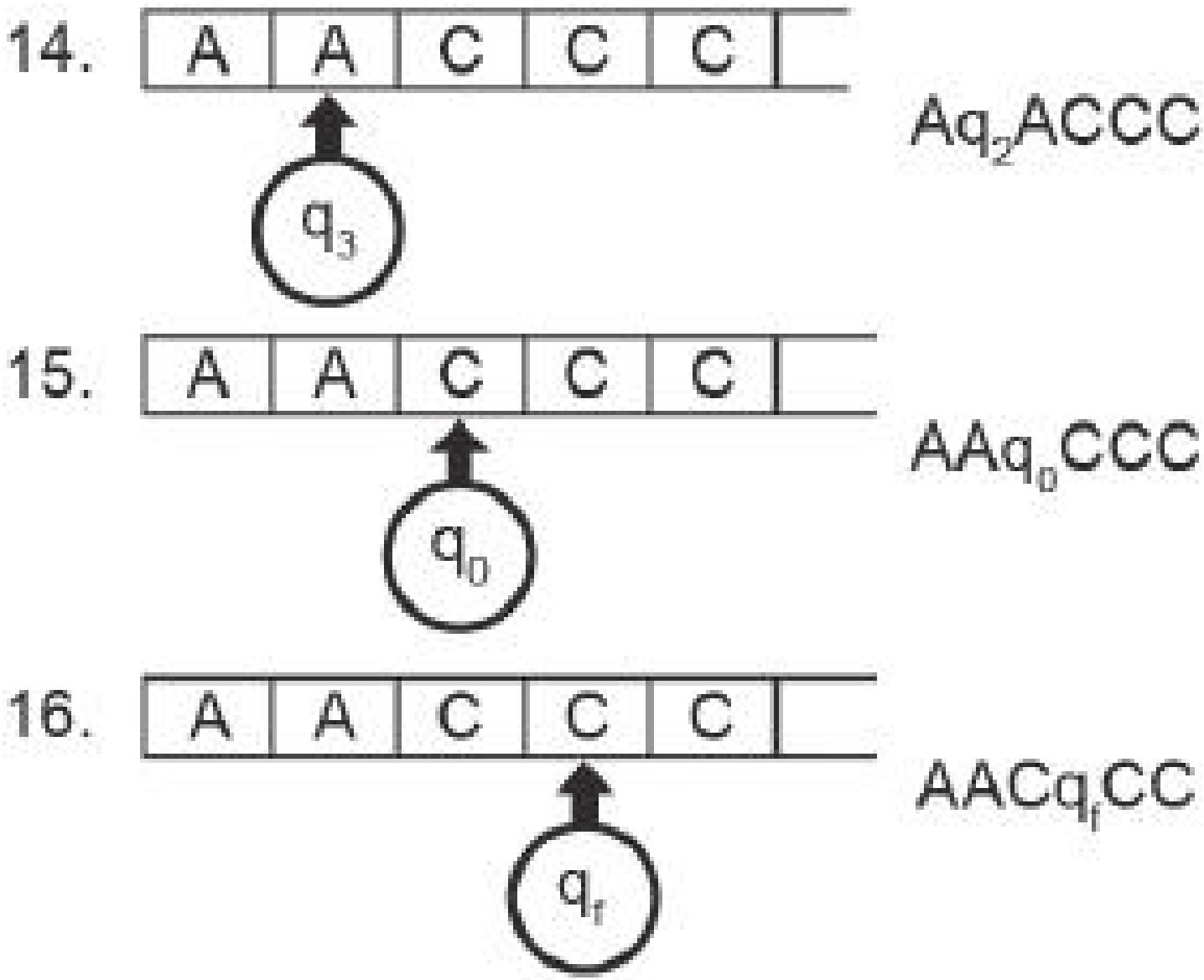
AAq_1cCC

12. 

$AAcq_1CC$

13. 

AAq_2cCC



(fim)

- Formalmente, uma MT possui alguns elementos em comum com um AF:
  - Um conjunto finito Q de estados
  - Um alfabeto de entrada Σ, sobre o qual as cadeias a serem reconhecidas estão definidas
  - Um conjunto $F \subseteq Q$ de estados finais
  - $q_0 \in Q$: estado inicial

- Como a MT possui uma fita que pode ser lida e escrita, ela possui os seguintes elementos diferentes de um AF:
  - Um conjunto finito Γ de símbolos que podem ser escritos na fita
  - $B \in \Gamma$, o símbolo 'branco' tal que $B \notin \Sigma$ e B não pode ser escrito na fita
  - A fita é infinita para a direita e é inicializada com a palavra a ser reconhecida seguida de infinitos brancos

conjunto finito de símbolos que podem ser escritos na fita

alfabeto de entrada

(continua)

(continuação)

- δ : uma função de $Q \times \Gamma$ em $\wp\,(Q \times (\Gamma - \{B\}) \times \{\leftarrow, \rightarrow\})$
  - Um estado e um símbolo sendo lido da fita podem estar associados a um conjunto de transições
  - Se este conjunto possuir sempre no máximo uma transição, dizemos que a MT é determinística
  - Cada transição pertence a $Q \times (\Gamma - \{B\}) \times \{\leftarrow, \rightarrow\}$,
    - portanto, possui um estado (estado destino), um símbolo não branco da fita (símbolo que irá sobrescrever o símbolo lido), e uma direção (esquerda ou direita) para a qual a cabeça de leitura irá se mover
  - Caso a cabeça se mova à esquerda do limite inicial da fita nada acontece, a máquina não vai para uma nova configuração

Exemplo - elementos constituintes

- $Q = \{ q_0 , q_1 , q_2 , q_3 , q_f \}$
  $\Sigma = \{ a , c \}$
  $F = \{ q_f \}$

AF do slide 13

- q_0 é o estado inicial

- $\Gamma = \{ a , c , a , C , B \}$

- B é o símbolo branco

(continua)

Exemplo (continuação)

- $\delta(q_0 , a) = \{(q_1 , A , \rightarrow)\}$
- $\delta(q_0 , C) = \{(q_f , C , \rightarrow)\}$
- $\delta(q_1 , a) = \{(q_1 , a , \rightarrow)\}$
- $\delta(q_1 , c) = \{(q_1 , c, \rightarrow)\}$
- $\delta(q_1 , B) = \{(q_2 , C , \leftarrow)\}$
- $\delta(q_1 , C) = \{(q_2 , C , \leftarrow)\}$
- $\delta(q_2 , c) = \{(q_3 , C , \leftarrow)\}$
- $\delta(q_3 , a) = \{(q_3 , a, \leftarrow)\}$
- $\delta(q_3 , c) = \{(q_3 , c, \leftarrow)\}$
- $\delta(q_3 , A) = \{(q_0 , A , \rightarrow)\}$

regras de transição

Exemplo (continuação)

- Alternativamente, para simplificar a notação, representamos a função de transição δ como quíntuplas pertencentes a

$$Q \times \Gamma \times Q \times \Gamma \times \{\rightarrow, \leftarrow\}$$

- Onde $\langle q, A, r, B, \rightarrow \rangle$ significa que $(r, B, \rightarrow) \in \delta(q, A)$
- A função δ do autômato anterior pode ser representada pelas quíntuplas:
  - $\langle q_0, a, q_1, A, \rightarrow \rangle$; $\langle q_0, C, q_f, C, \rightarrow \rangle$;
    $\langle q_1, c, q_1, c, \rightarrow \rangle$; $\langle q_1, B, q_2, B, \leftarrow \rangle$;
    $\langle q_1, C, q_2, C, \leftarrow \rangle$; $\langle q_2, c, q_3, C, \leftarrow \rangle$;
    $\langle q_3, a, q_3, a, \leftarrow \rangle$; $\langle q_3, c, q_3, c, \leftarrow \rangle$;
    $\langle q_3, A, q_0, A, \rightarrow \rangle$

Definição

- Formalmente, definimos uma MT M como uma tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, onde os componentes foram descritos anteriormente

- Seja uma MT que está em uma configuração $\alpha q a \beta$, possui a transição
  $(r, x, \rightarrow) \in \delta(q, a)$; onde $q, r \in Q$; $a, x \in \Gamma$; $\alpha, \beta \in \Gamma^*$;
  e se move para a configuração $\alpha x r \beta$

- A MT muda de uma configuração para outra seguindo uma regra de transição que inclui mudar de estado, escrever um símbolo e mover a cabeça de leitura/escrita
  - Neste caso descrevemos como $\alpha q a \beta \vdash \alpha x r \beta$

(continua)

- $(r, x, \rightarrow) \in \delta(q, a)$; onde $q, r \in Q$; $a, x \in \Gamma$; $\alpha, \beta \in \Gamma^*$
  - Configuração atual
    - α e β são sequências de símbolos na fita
    - q é o estado atual da máquina
    - a é o símbolo na célula atual
  - Transição
    - Existe a transição $(r, x, \rightarrow)$ na função δ, que diz que
      - Se a máquina está no estado q e lê o símbolo a,
      - ela escreve o símbolo x no lugar de a,
      - move a cabeça de leitura/escrita para a direita ($\rightarrow$),
      - e muda para o estado r
  - Nova configuração ($\alpha x r \beta$)
    - α permanece igual,
    - a é substituído por x,
    - o estado muda para r,
    - e a cabeça de leitura/escrita move-se para a célula à direita de x

- De forma análoga, se a máquina está em uma configuração $\alpha bqa\beta$, possui a transição
  $(r, x, \leftarrow) \in \delta(q, a)$; onde $q, r \in Q$; $a, x \in \Gamma$; $\alpha, \beta \in \Gamma^*$;
  e se move para a configuração $\alpha rbx\beta$

- Neste caso, escrevemos $\alpha bqa\beta \vdash \alpha rbx\beta$

- Se a MT não pode se mover em uma dada configuração, dizemos que a máquina parou nesta configuração

- Chamamos $\vdash^*$ à aplicação de 0 ou mais vezes da relação $\vdash$

- A MT $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ reconhece uma sentença w se, começando do estado inicial q_0, e lendo a sentença w a partir da fita, a máquina pode fazer uma série de transições, denotadas por $\vdash^*$, até alcançar uma configuração $a\, q_f\, \beta$, onde q_f é um estado de aceitação (ou estado final) pertencente ao conjunto F
  - ou, $q_0\, w \vdash^* \alpha\, q_f\, \beta$ onde $q_f \in F$, e a MT para na configuração $\alpha\, q_f\, \beta$

- A linguagem reconhecida pela MT é o conjunto de todas as sentenças reconhecidas pela MT

- Dada uma MT $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, definimos a linguagem reconhecida pela MT como:

$$T(M) = \{ w \in \Sigma^* \mid \exists\, q_f \in F ,\ q_0\, w \vdash^* \alpha\, q_f\, \beta \ \text{ e } \ M \text{ para em } \alpha\, q_f\, \beta \}$$

  - A linguagem reconhecida pela Máquina de Turing M, denotada por T(M), é o conjunto de todas as strings w pertencentes a Σ^* (ou seja, todas as strings de entrada possíveis), tal que existe um estado final q_f pertencente a F, e a partir da configuração inicial q_0 w, a MT transiciona em zero ou mais passos para uma configuração $\alpha\, q_f\, \beta$, onde α e β são strings de símbolos da fita, e a MT para nessa configuração $\alpha\, q_f\, \beta$

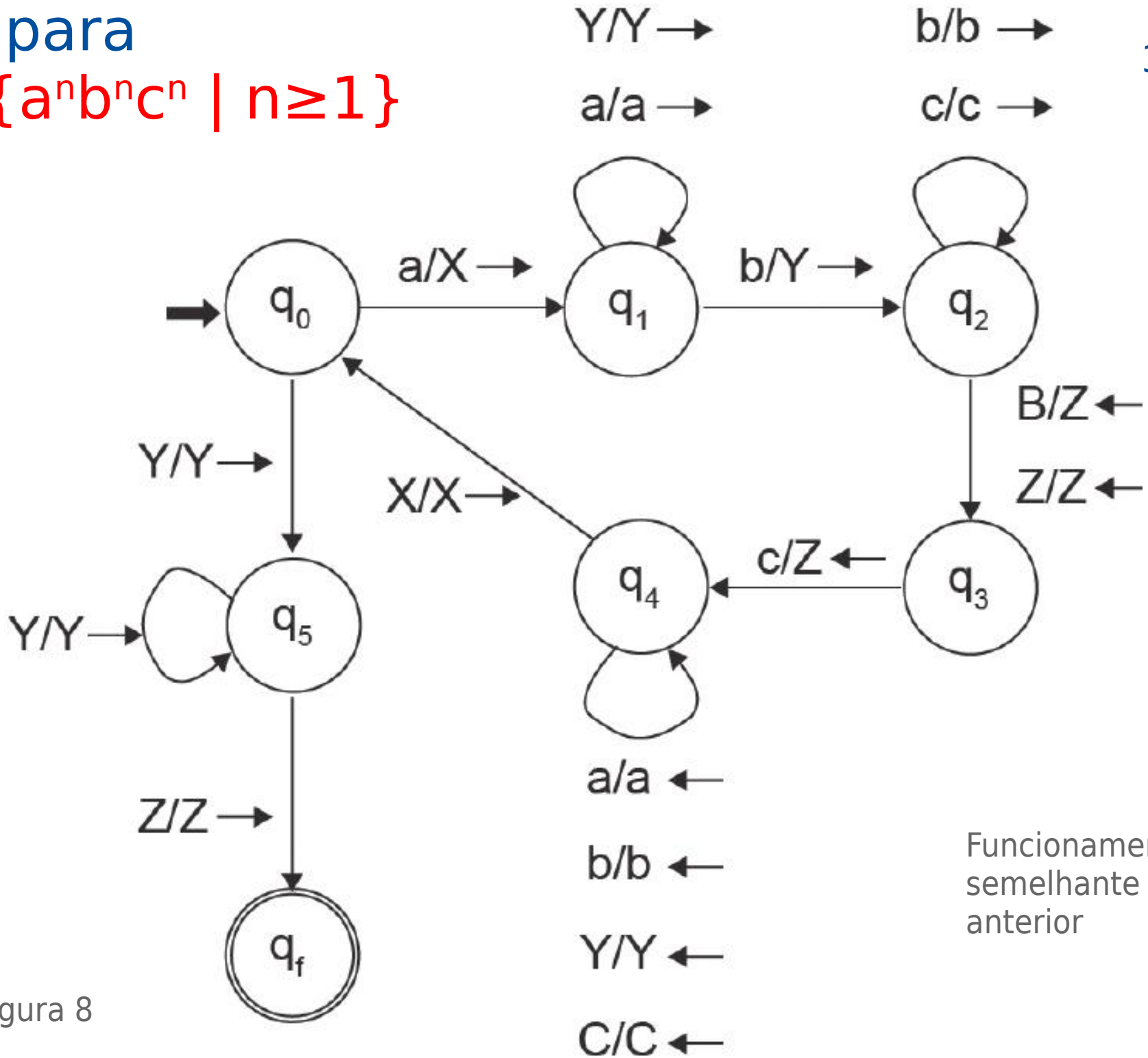
- Segundo a definição, para uma MT não determinística (MTND), basta um dos caminhos possíveis fazer a MT parar em um estado final para que a MT reconheça a cadeia

- Na MTND, em vez de especificar uma única ação para cada combinação de estado e símbolo, a função de transição pode especificar um conjunto de ações possíveis

- Os exemplos vistos até aqui poderiam ser reconhecidos por um autômato com pilha

- A figura do próximo slide apresenta uma MT que reconhece a linguagem $L = \{a^n b^n c^n \mid n \geq 1\}$ (SC)

MT para
$L=\{a^n b^n c^n \mid n \geq 1\}$



Funcionamento semelhante à anterior

Figura 8

MT para $L=\{a^n b^n c^n \mid n \geq 1\}$

- O AF da Fig. 8 é construído a partir da Fig. 6
  - q5 verifica se sobraram símbolos entre Y e Z
    - serve como uma verificação final para assegurar que todos os símbolos foram processados e marcados corretamente, sem sobras que invalidariam a cadeia
      - Na Fig. 6 a MT não precisa de um estado adicional como q_5 porque a estrutura mais simples da linguagem permite que a verificação seja feita implicitamente durante o processo de marcação
        - marcação é técnica de substituir símbolos na fita por símbolos especiais (marcadores) para indicar que esses símbolos já foram processados
          - Ex.: a MT substitui os símbolos originais (como a, b, c) por símbolos especiais (como X, Y, Z) para indicar que eles foram processados
            - A fita contém a cadeia aaabbb. Fita após as marcações: XXXYYY. No final se a não houver sobras, a MT aceita a cadeia.

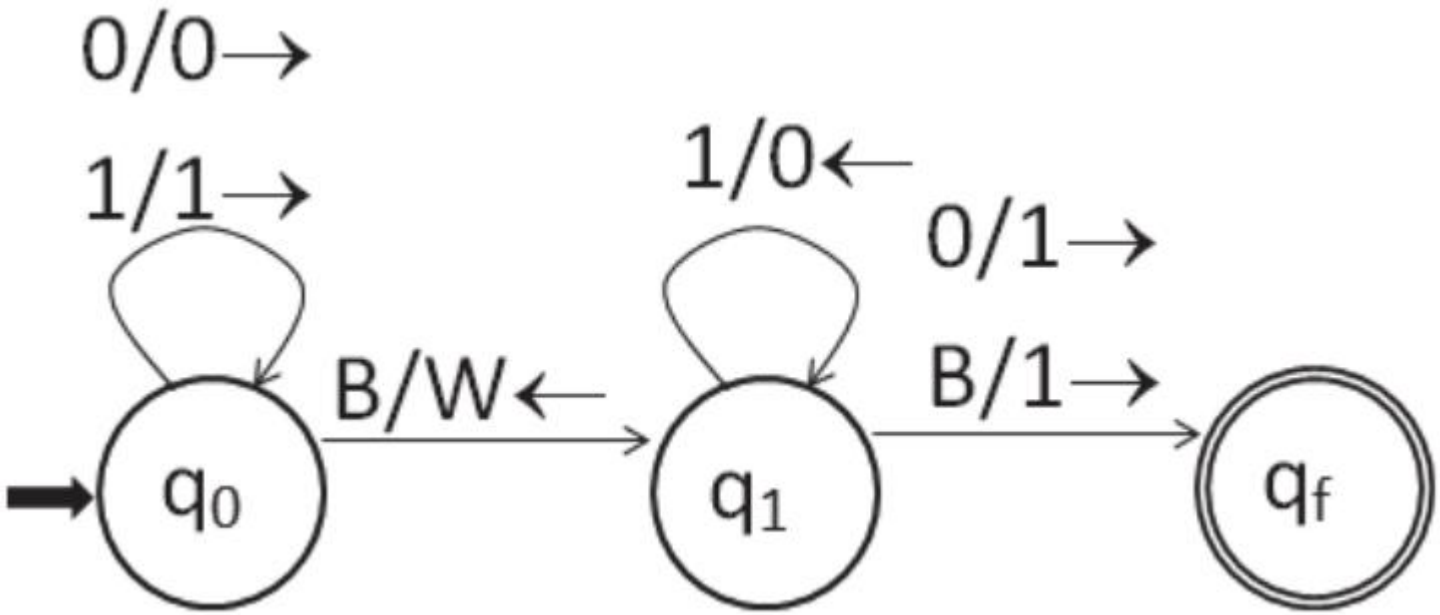
MT para $L=\{a^n b^n c^n \mid n \geq 1\}$

- O estado q_3 da primeira figura não é necessário na segunda figura porque a lógica de processamento e marcação é diferente e mais complexa na segunda MT

- A substituição de a por X, b por Y, c por Z e B por Z na segunda figura, garante que todos os símbolos sejam processados corretamente e que a cadeia pertença à $L=\{a^n b^n c^n \mid n \geq 1\}$
  - Ex.: se a fita contém aaabbbcccB , após processar todos os símbolos, a fita ficará XXXYYYZZZZ

MT implementando função

- Podemos usar um MT para implementar uma função,
  - ao invés de reconhecer uma linguagem
- Ex.: dada uma MT com entrada sendo uma cadeia sobre {0,1}, representando um nº na base 2, a MT soma 1 a este nº
  - Isso significa que se a máquina começa com a representação binária do nº n na fita, ela deve parar contendo na fita a representação binária de n+1
  - Como não permitimos escrever o símbolo branco, B, vamos permitir que a fita tenha um caractere não branco W após o fim da representação binária de n+1



- Para simplificar a máquina, supomos neste caso particular a fita infinita para ambos os lados

- Não seria necessário usar um estado final, porque não queremos reconhecer uma linguagem
- "As MT utilizam as células da fita para processar a entrada, e o término da computação é determinado pelo estado atual da MT e pelos símbolos lidos na fita
  - Uma MT com fita limitada, também conhecida como Autômato Linearmente Limitado (ALL), é uma MT cuja fita é limitada por uma função linear do tamanho da entrada, em vez de ser infinita
- Uma linguagem pode ser reconhecida por uma MT com fita limitada se, e somente se, é uma linguagem sensível ao contexto

Máquinas de Turing e Recursividade

Eduardo Furlan Miranda

Baseado em: GARCIA, A. de V.; HAEUSLER, E. H.
Linguagens Formais e Autômatos. Londrina: EDA, 2017.

Máquina de Turing (MT)

- 1900: desafio intelectual do programa de Hilbert
  - Provar a consistência e decidibilidade da matemática
    - Necessidade de formalizar operações efetivas sobre objetos concretos (números naturais)
      - Tentativa de encontrar um conjunto completo e consistente de axiomas para toda a matemática
- 1931: prova da indecidibilidade, por Gödel
  - Existência de enunciados matemáticos indecidíveis
- 1936: cálculo-λ, por Church
  - Conceito de funções anônimas (funções sem nome)
    - Equivalente à MT em termos de capacidade computacional,
      - Influenciou linguagens de programação funcionais, como Lisp

MT

- 1937: MT, por Turing
  - Modelo matemático (teórico) para estudar números computáveis
  - Conceito de "máquinas automáticas"
    - uma máquina com uma fita para leitura e escrita de caracteres e uma memória de estados finita
  - Equivalência com o conceito de computável de Church

Tese de Church-Turing

- Qualquer função que pode ser calculada por um procedimento efetivo (ou algoritmo), pode ser calculada por uma MT
- A MT se torna um modelo fundamental para a teoria da computação
  - Influenciando o desenvolvimento de computadores e linguagens de programação

Tese = “funciona”. Amplamente aceito, porém não é uma verdade matemática no mesmo sentido de um teorema.

Computabilidade

- É natural e não possui uma definição definitiva
  - Similar a outros conceitos fundamentais como tempo, espaço, movimento, força e vida
- A Computação é tanto Ciência, quanto Física ou Biologia
  - Sujeita a construção de teorias potencialmente refutáveis
- A busca por um modelo formal do que é computável culminou na Tese de Church-Turing,
  - estabelecendo um fundamento sólido para a teoria da computação
    - e mostrando a intersecção entre matemática e ciência empírica

se baseia em observações,
experimentações e experiências diretas

Parada

- Problema da Parada (Halting Problem)
  - Turing provou a indecidibilidade do problema da parada :
    - questiona se existe um algoritmo que pode determinar se um programa de computador, dado uma entrada específica, eventualmente irá parar (terminar) ou continuará a rodar indefinidamente

- Turing e Church
  - Não existe um processo algorítmico geral para determinar a veracidade de proposições matemáticas
    - Mostra as limitações de sistemas formais e de algoritmos

MT Universal (MTU)

- Turing é quem de fato demonstra matematicamente que é possível construir uma máquina programável
  - que lê programas e os executa
  - antes dos primeiros computadores
- Cada MT realiza uma só tarefa
- MT Universal (MTU)
  - Capaz de simular qualquer outra MT
    - Ilustra o conceito de um interpretador de linguagem de programação

MTs e linguagens

- Cada MT aceita uma única linguagem, ou seja,
  - existe um mapeamento entre a MT e a linguagem que ela aceita
- É possível definir uma quantidade infinita de MT que aceitam a mesma linguagem, ao adicionar instruções inúteis que nunca serão executadas
- Quando uma MT nunca para (nunca chega a um estado de aceitação), independentemente da cadeia de entrada, ou seja, não consegue aceitar nenhuma cadeia de símbolos,
  - dizemos que ela reconhece a linguagem vazia porque não possui nenhuma cadeia

Linguagens e limitações de uma MT

- Máquina que aceita Σ^* : é a máquina que para em um estado final, para todas as cadeias em Σ^* ,
  - onde Σ é o alfabeto de entrada da máquina.
    - Σ^* representa o conjunto de todas as possíveis cadeias que podem ser formadas usando os símbolos do alfabeto Σ, incluindo a cadeia vazia (ε)
- Toda MT define uma linguagem, e existem infinitas máquinas que definem a mesma linguagem
  - o inverso não é verdadeiro
- Existem muito mais linguagens não aceitas por MT do que linguagens aceitas

* fecho de Kleene

Recursividade

- Capacidade de uma linguagem ser decidida por uma MT que sempre para,
  - retornando "sim" para cadeias que pertencem à linguagem
  - e "não" para as que não pertencem
- Uma linguagem é recursiva se existe uma função característica Turing-computável que mapeia cada cadeia da linguagem em 1 (se pertence) ou 0 (se não pertence)
- Essencialmente, uma linguagem é recursiva se existe um algoritmo que sempre consegue determinar se uma dada cadeia faz parte dela ou não,
  - o que garante que o processo de decisão sempre termina com uma resposta

Recursividade

- A recursividade em MTs está relacionada à capacidade de computação e à decidibilidade de problemas, ou seja,
  - a existência de um algoritmo que sempre consegue determinar se uma dada cadeia pertence ou não a uma linguagem
- A recursividade em linguagens de programação é uma técnica utilizada para implementar algoritmos
  - onde uma função chama a si mesma repetidamente

Decidível e semidecidível

- Linguagens recursivamente enumeráveis (RE) (semidecidíveis)
  - são aquelas para as quais existe uma MT que aceita todas as cadeias da linguagem, mas pode não rejeitar cadeias fora da linguagem
    - pode não parar
- Linguagens recursivas (decidíveis)
  - são um subconjunto das linguagens RE , onde a MT sempre para e decide corretamente se uma cadeia pertence ou não à linguagem
    - sempre para e aceita ou rejeita

Recursivamente Enumerável (RE)

- Uma linguagem $L \subseteq \Sigma^*$ é RE se, e somente se, existe uma MT M, tal que, $L(M) = L$
  - Dada uma L que é RE, então pode-se construir uma MT que aceita exatamente essa linguagem
  - L : conjunto de cadeias formadas por símbolos de um alfabeto
  - $L(M)$: a linguagem aceita pela MT M
    - conjunto de todas as cadeias w para as quais a MT M, ao receber w como entrada, para em um estado final
  - $L \subseteq \Sigma^*$: a linguagem L é um subconjunto de Σ^*

RE = semidecidível, pode não parar

MTs e linguagens RE

- Para demonstrar que uma L é RE ,
  - basta construir uma MT que a reconheça ,
    - que pare em um estado de aceitação para as entradas pertencentes à linguagem ;
      - não é necessário que a MT sempre pare

- Uma MT pode reconhecer strings que não são codificações válidas de MTs ,
  - verificando se seguem a sintaxe definida para codificar MTs .
    - Se não obedecer a essa sintaxe (por exemplo, se tiver um formato inválido, símbolos não permitidos ou uma estrutura inconsistente) ,
      - a MT pode imediatamente rejeitá-la (parando em um estado não final), ou entrar em um estado de erro

transita para um estado específico que representa a detecção de uma entrada inválida

LP

- Linguagem Paradoxal (LP)
  - Definida como o conjunto de MT que não aceitam a si mesmas
  - Formalmente, pode ser expressa como

    LP = { M | M é uma MT e M não aceita M}
  - Não é recursivamente enumerável, devido a um argumento de contradição
    - O argumento de contradição mostra que a existência de uma MT que reconhece a LP leva a uma situação impossível, onde a MT teria que aceitar e não aceitar sua própria codificação ao mesmo tempo
      - Isso prova que uma tal MT não pode existir e, consequentemente, que a LP não é recursivamente enumerável

- RE = semidecidível, pode não parar
- não é RE : não pertence à classe de linguagens que podem ser reconhecidas por uma MT

Autoaceitação

- Uma MT "aceita a si mesma" se, ao executar com sua própria descrição codificada como entrada, ela atinge o estado final
  - MTs podem ser representadas como strings finitas, permitindo que sejam usadas como entrada para outras MTs

- $P = \{M \mid M \in \Sigma_{Turing} \text{ e } M \text{ aceita } M\}$
  - P : linguagem das MT que aceitam a si mesmas
  - M : uma Máquina de Turing
  - Σ_{Turing} : conjunto de todas as codificações de MTs
  - M aceita M : a MT M, ao receber sua própria codificação como entrada, para em um estado final

Definições de P e ¬P

¬ = complemento

- P = {M | M aceita M}
  - conjunto que contém todas as MT que aceitam sua própria codificação como entrada
- ¬P = {M | M não aceita M}
  - o complemento de P é o conjunto de todas as MT que não aceitam sua própria codificação como entrada

complemento do conjunto P

O ¬P é recursivamente enumerável

¬ = complemento

- ¬P é um conjunto recursivamente enumerável
  - Isso significa que existe uma MT que pode listar todos os elementos de ¬P
    - Em outras palavras, existe uma MT que aceita uma máquina M se esta não aceita sua própria codificação

- Implicação
  - Se uma MT M rejeita a si mesma como entrada, isso significa que M pertence ao conjunto ¬P
    - A MT que reconhece ¬P aceitaria então M
  - O conjunto das cadeias que não são MT é facilmente aceito por uma MT,
    - significando que é relativamente simples construir uma MT que reconheça todas as cadeias de símbolos que não representam a codificação de uma MT válida

RE = semidecidível, pode não parar

Reconhecer

- Existem problemas para os quais podemos verificar uma solução (reconhecer),
  - mas não podemos determinar se uma solução existe (decidir)

- Verificar (reconhecer)
  - Uma MT pode ser programada para reconhecer soluções
    - Isso significa que, se fornecermos uma cadeia (solução) à MT, ela pode executar seu conjunto de instruções e determinar se essa cadeia é uma solução válida para o problema
      - Se for, a MT aceitará essa cadeia, entrando em um estado de aceitação

chegou a um estado final, indicando que a entrada foi reconhecida como válida

Decidir

- Determinar (decidir)
  - Significa determinar se existe uma solução para qualquer entrada fornecida
    - Para alguns problemas, não há uma maneira garantida de uma MT descobrir se uma solução existe ou não
      - A MT pode não parar nunca ou pode entrar em um ciclo infinito sem encontrar a solução

Função característica F_L

- É implementada rodando 2 MTs em paralelo
  - uma que aceita todas as cadeias de uma linguagem L ,
  - e outra que aceita todas as cadeias que não estão em L
    - (ou seja, o complemento $\overline{L}$)
- A função retorna 1 se a cadeia pertence a L, e 0 caso contrário
- Para uma linguagem $L \subseteq \Sigma^*$, L é recursiva (decidível) se, e somente se, tanto L quanto seu complemento $\overline{L}$ são recursivamente enumeráveis (semidecidíveis)
  - Isso ocorre porque, se L e $\overline{L}$ são RE, podemos usar a execução em paralelo das duas MTs para garantir que a MT principal sempre pare e decida se uma cadeia pertence a L ou a $\overline{L}$

RE = semidecidível, pode não parar

Recursividade e complemento

- Se L é recursiva, então $\overline{L}$ também é recursiva
  - Isso porque a MT que decide L pode ser adaptada para decidir $\overline{L}$ simplesmente invertendo suas respostas (aceitar quando L rejeita e vice-versa)

- A execução em paralelo é essencial nesse contexto porque garante que a MT principal sempre pare, mesmo que uma das MTs individuais (para L ou $\overline{L}$) não pare para certas entradas
  - Dessa forma, a MT principal consegue decidir L de maneira eficiente e completa

O Problema da Parada Paradoxal

- Questiona se uma MT para quando recebe seu próprio código como entrada : o problema é indecidível
  - Implica que a linguagem das MTs que não se aceitam (e consequentemente, as que se aceitam) não é recursiva
    - Para uma linguagem ser recursiva, deve existir uma MT que sempre decide (para e dá uma resposta) se uma cadeia pertence ou não à linguagem

- A tentativa de criar um algoritmo para determinar se uma MT para com seu próprio código leva a uma contradição lógica,
  - demonstrando a indecidibilidade desse problema e
    - a não recursividade das linguagens relacionadas

Indecidibilidade

- A indecidibilidade e a impossibilidade de detecção de comportamentos em programas:
  - Um problema relevante com parada de máquinas é o problema da parada L_{Para} que é a linguagem das cadeias M$d , onde M é uma MT , d uma cadeia qualquer, e M para (aceitando portanto) quando tem d por entrada
  - É indecidível determinar se M$d pertence a L_{Para}
    - Não existe um algoritmo geral que determine se uma MT M para para uma entrada arbitrária d

L_{Para} é um conjunto (ou linguagem) que contém todas as possíveis combinações de uma MT e uma entrada d para a qual essa MT para

M = MT M
$ = símbolo separador
d = entrada
É uma notação que indica uma máquina de Turing específica (M) e os dados (d) que ela processará

Teorema de Rice

- Demonstra que não é possível criar um algoritmo geral para verificar se uma MT aceita uma propriedade não trivial, o que implica que diversos problemas relacionados ao comportamento das MTs são indecidíveis
  - Propriedades não triviais são características de cadeias sobre um alfabeto que não são universais (satisfeitas por todas as cadeias) nem impossíveis (não satisfeitas por nenhuma cadeia)
    - Elas se aplicam a algumas cadeias, mas não a todas
      - Exemplos incluem ter comprimento par, ser balanceada, ou começar e terminar com símbolos específicos
  - Propriedades triviais são aquelas que são satisfeitas por todas as cadeias (como a linguagem Σ^*) ou por nenhuma cadeia (como a linguagem vazia $\emptyset$)

GI e linguagens RE

- Gramáticas Irrestritas (GI) geram exatamente as linguagens recursivamente enumeráveis (RE)
- Uma máquina de Turing (MT) pode simular as derivações de uma GI, aceitando uma cadeia se ela for gerada pela gramática
- Essa simulação demonstra a equivalência entre GI e linguagens RE

RE = semidecidível, pode não parar

GI e linguagens RE

- A MT aceita uma cadeia se, e somente se, ela puder ser derivada pela gramática
- Caso contrário, a MT entra em loop infinito
- Essa característica define as linguagens recursivamente enumeráveis: uma linguagem é RE se existe uma MT que a aceita dessa forma

GI e MTs equivalentes

- A recíproca também é verdadeira: dada uma MT, podemos construir uma GI que gera exatamente as cadeias aceitas pela MT
- A ideia é simular as configurações da MT usando as regras da GI
- Codificamos as configurações como cadeias $\alpha Q \beta$, onde:
  - α é a parte da fita à esquerda da cabeça de leitura/escrita
  - Q é o estado atual da MT
  - β é a parte da fita à direita da cabeça de leitura/escrita (incluindo o símbolo sob a cabeça)

(continua)

GI e MT

- Para simular as transições da MT, a GI possui regras da forma $Q\sigma \rightarrow \sigma' Q'$, correspondendo às quíntuplas de transição da MT

  $\langle Q, \sigma, Q', \sigma', \rightarrow \rangle$

- A configuração inicial é gerada por regras que produzem cadeias da forma $\alpha\$Q_0\alpha$, onde α é qualquer cadeia do alfabeto da MT e Q_0 é o estado inicial

- Assim, a GI gera uma cadeia se, e somente se, a MT pode alcançar a configuração correspondente

GI e MT

- A gramática simula a aceitação da MT através de regras como $Qf\sigma \rightarrow Qf$, $\sigma Qf \rightarrow Qf$ e $\$Qf \rightarrow \varepsilon$, garantindo que a derivação termina quando a MT alcança um estado final

- Assim, uma cadeia é gerada pela GI se, e somente se, a MT a aceita

- Essa correspondência biunívoca demonstra a equivalência entre MTs e GIs
  - Para toda gramática irrestrita G, existe uma MT M tal que L(M) = L(G)
  - Para toda MT M, existe uma gramática irrestrita G tal que L(M) = L(G)